

Nodka installation and considerations

Frits Hamers

October 23, 2024

4

0.1 Revision history

Revision	Date	Change	Name
001	2024-08-07	First draft proposal	Frits
002	2024-09-18	Added login configuration Corrected firewall settings Added Network settings Added missing dkms configuration Added PyCyphal install instructions	Frits
003	2024-09-20	Corrected some typos Added PyCyphal environmental variables Added missing dkms configuration Corrected rev 002 change-log	Frits
004	2024-10-04	Added additional PyCyphal environmental variables Added CANFD configuration	Frits
005	2024-10-16	Updated CANFD configuration by adding Sample Point = 80%	Frits
006	2024-10-23	Updated environmental variables Updated CAN FD configuration to add SyncJumpWidth Added PyDsdI installation instructions	Frits

5

6

0.2 Audience

001	Tomy	Jattinder			
002	Tomy	Jattinder			
003	Tomy	Jattinder			
004	Tomy	Jattinder			
005	Tomy	Jattinder			
006	Tomy	Jattinder			

7

Contents

8

9	0.1 Revision history	2
10	0.2 Audience	2
11	1 Introduction	7
12	1.1 scope	7
13	1.1.1 environment	7
14	2 References	8
15	2.1 Referenced documentation	8
16	2.2 References files	8
17	2.3 Markings for review purpose	8
18	2.4 Terminology	8
19	2.5 Abbreviations	9
20	3 Hardware	10
21	4 Operating System	11
22	4.1 BIOS settings	11
23	4.2 Installation procedure	11
24	4.2.1 Set up data encryption	11
25	4.2.2 OS configuration	11
26	4.2.3 Data decryption at boot	11
27	4.2.4 auditd	13
28	4.2.5 Login configuration	13
29	5 Software packages	15
30	5.1 CURL	15
31	5.2 firewall	15
32	5.2.1 firewall configuration	15
33	5.2.2 Firewall documentation	16
34	5.2.3 Rerouting RS485 ports	16
35	5.3 CAN FD driver	16
36	5.3.1 Set-up a key for a signed kernel module	16
37	5.3.2 build a signed kernel module	17
38	5.4 Webmin	17
39	5.4.1 Install the Webmin software packages	17
40	5.4.2 Use Webmin	17

41	6 Install Configuration and additional installation	18
42	6.1 Configuration of the network devices	18
43	6.1.1 Network device 1	18
44	6.1.2 Network device 2	18
45	6.1.3 Network device 3	19
46	6.1.4 CAN FD 0	19
47	6.1.5 CAN FD 1	20
48	6.2 CodeSys	20
49	6.3 Cyphal	20
50	6.3.1 Pyton 3	20
51	6.3.2 PyCyphal	20
52	6.3.3 PyDsdI	21

53 List of Tables

54	2.1 Terminology	9
55	2.2 Terminology	9

List of Figures

1. Introduction

This document describes how to get Webmin running on a Debian 10 based AX as part of a feasibility study. The contents of this document are for development and demo purpose only and should **not** be used in a production environment.

1.1 scope

The scope of the feasibility study is to investigate weather the utilisation of Webmin will enable compliance with the cyber security standard [**doc:1**].

1.1.1 environment

Having a Linux environment as a target, Linux based tools are used. MS Windows equivalents are outside the scope of this study.

2. References

2.1 Referenced documentation

[1.1] *IEC/ISA 62443-3-3*

[1.2] *Firewalld official site*

[1.3] *Webmin official site*

[1.4] *LUKS and TPM auto decryption*

2.2 References files

[2.1] *Some file.so*

2.3 Markings for review purpose

Proposed change

Revision required

To be defined

Delta compared with the previous revision

A comment addressed to the reviewer

2.4 Terminology

Term	Description of the used term.
industrial automation and control system	collection of personnel, hardware, software and policies involved in the operation of the industrial process and that can affect or influence its safe, secure and reliable operation.

mobile device	intelligent electronic device intended for use while mobile Note to entry: Examples of mobile devices include laptop computers (depending on their usage, laptops can also be considered “portable devices”), mobile robots, smart phones, hand-held programmers, tablet computers and personal digital assistants.
non-repudiation	ability to prove the occurrence of a claimed event or action and its originating entities. Note to entry: The purpose of non-repudiation is to resolve disputes about the occurrence or non-occurrence of the event or action and involvement of entities in the event.
portable device	intelligent electronic device intended to be used in more than one physical location, but not intended for use while in transport between locations. Note to entry: Examples of portable devices include laptop computers (depending on their usage, laptops can also be considered “mobile devices”), maintenance workstations and analysers.
role	set of connected behaviours, privileges and obligations associated with all users (humans, software processes or devices) of an IACS Note to entry: The privileges to perform certain operations are assigned to specific roles.

Table 2.1: Terminology

2.5 Abbreviations

82

Abbreviation	Full form
DC	Data confidentiality
EFI	Extensible Firmware Interface
FR	Foundational requirement
IAC	Identification and authentication control
IACS	Industrial automation and control system(s)
LAN	Local Area Network
LVM	Logical Volume Manager
OS	Operating System
PC	Personal Computer
RA	Resource availability
RDF	Restricted data flow
SI	System integrity
SR	System requirement
ssh	Secure Shell
TRE	Timely response to events
UC	Use control
UEFI	Unified Extensible Firmware Interface
USB	Universal Serial Bus

Table 2.2: Terminology

3. Hardware

4. Operating System

4.1 BIOS settings

4.2 Installation procedure

Install minimum os and do not enable the root user

4.2.1 Set up data encryption

Use the partition manager to create an encrypted LVM. Choose for separated home, var and tmp partitions. provide a passphrase with a minimum of 20 characters. This passphrase is later required at boot time.

4.2.2 OS configuration

Some additional packages are required. To be able to install these packaged, the repository from the install media should be disabled.

```
$ sudo vi /etc/apt/sources.list
```

and comment out the line beginning with "deb cdrom:"

A text editor is needed to add and modify the configuration files. In this case "vim" is used, however "nano" can be used in stead when preferred.

```
$ apt install vim
```

4.2.3 Data decryption at boot

The following is taken from the article [**LUKS and TPM auto decryption**] referred in [2.1](#).

O.S. Partition First it should be determined on which partition the O.S. is installed. It is most likely that the hard drive / solid state drive is divided into three partitions. /boot/efi /boot and the partition containing the LVM. The following command can be used to obtain an overview of the available storage devices and their partitions:

```
$ sudo fdisk -l
```

This might return something like:

```

Disk /dev/sda: 119.24 GiB, 128035676160 bytes, 250069680 sectors
Disk model: FORESEE 128GB SS
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: F718A534-FEC4-4A32-9CAA-A26CD5677E3B

Device Start End Sectors Size Type
/dev/sda1 2048 1050623 1048576 512M EFI System
/dev/sda2 1050624 2050047 999424 488M Linux filesystem
/dev/sda3 2050048 250068991 248018944 118.3G Linux filesystem

```

Here it's almost clear that /dev/sda1 is mounted to /boot/efi and /dev/sda2 to /boot. This means that /dev/sda3 contains the LVM with the O.S. on it. The command disk free can be use to validate that.

```
$ df
```

And this returns something like:

```

Filesystem 1K-blocks Used Available Use% Mounted on
devtmpfs 4096 0 4096 0% /dev
tmpfs 3985040 3156 3981884 1% /dev/shm
tmpfs 1594016 9168 1584848 1% /run
tmpfs 5120 0 5120 0% /run/lock
/dev/mapper/battcash--vg-root 22699144 2310164 19210564 11% /
/dev/mapper/battcash--vg-home 88252376 41132 83682300 1% /home
/dev/mapper/battcash--vg-var 7841360 308660 7112860 5% /var
/dev/mapper/battcash--vg-tmp 1370072 56 1282160 1% /tmp
/dev/sda2 466026 147752 293289 34% /boot
/dev/sda1 523244 11928 511316 3% /boot/efi
tmpfs 797008 0 797008 0% /run/user/1000

```

And this confirms that /dev/sda3 contains the encrypted LVM.

Required packages The following commands should be executed once to obtain the packages containing the required tools to enable automated decryption at boot.

```

$ sudo apt install tpm2-tools
$ sudo apt install dracut

```

Enrol TPM2 to the LVM and configure for auto decryption The partition containing the encrypted LVM can be enrolled for decryption. For more details, please refer to the source article.

```
$ sudo systemd-cryptenroll --wipe-slot=tpm2 --tpm2-device=auto --tpm2-  
→ pcrs=0+7 /dev/sda3
```

Dracut is used to generate the RAMdrive containing the kernel and should be aware of the automated decryption. This can be done by creating and edit the following file

```
$ sudo vim /etc/dracut.conf.d/tpm2.conf
```

and add the following line

```
add_dracutmodules+=("_tpm2-tss_crypt_")
```

Also the bootloader is involved and its configuration should be modified by editing

```
$ sudo vim /etc/default/grub
```

And add, or enable

```
GRUB_CMDLINE_LINUX="rd.auto rd.luks=1"
```

at the appropriate place.

Finally crypttab should be disabled

```
sudo vim /etc/crypttab
```

and the line containing the reference to the encrypted LVM should be commented out.

To apply the changes the following commands should be executed

```
$ sudo dracut -f  
$ sudo update-grub  
$ systemctl reboot
```

After reboot the tpm2 enrolled devices can be listed with:

```
$ sudo systemd-cryptenroll --tpm2-device=list
```

4.2.4 auditd

```
$ sudo apt install auditd
```

4.2.5 Login configuration

When a tty or ssh session is left idle, it should log off after a specified amount of time. This can be configured by changing the following configuration file:

```
$ sudo vim /etc/systemd/logind.conf
```

Uncomment the following line and replace "infinity" with a amount of seconds.

212
213
214

```
#StopIdleSessionSec=infinity
```

215 In this example an idle time of 5 minutes is applied.

216
217
218

```
StopIdleSessionSec=300
```

5. Software packages

5.1 CURL

```
$ sudo apt install curl
```

5.2 firewall

```
$ sudo apt install firewalld
```

5.2.1 firewall configuration

With the firewall up and running, the following commands should be executed to provide access for Webmin:

```
$ sudo firewall-cmd --permanent --zone=public --add-port=10000/tcp
```

To enable Codesys access some ports should be opened with the following commands:

```
$ sudo firewall-cmd --permanent --zone=public --add-port=1740-1743/udp
$ sudo firewall-cmd --permanent --zone=public --add-port=1217/tcp
$ sudo firewall-cmd --permanent --zone=public --add-port=8080/tcp
$ sudo firewall-cmd --permanent --zone=public --add-port=11740/tcp
$ sudo firewall-cmd --permanent --zone=public --add-port=443/tcp
$ sudo firewall-cmd --permanent --zone=public --add-port=4840/tcp
$ sudo firewall-cmd --permanent --zone=public --add-port=44818/tcp
$ sudo firewall-cmd --permanent --zone=public --add-port=2222/udp
```

sudo systemctl mask getty.target and

```
$ sudo firewall-cmd --reload
```

To check whether the new rule is applied, the following command can be used:

```
$ firewall-cmd --list-all
```

5.2.2 Firewall documentation

Check the reference to the [Firewalld official site](#) for more details.

5.2.3 Rerouting RS485 ports

```
$ sudo systemctl mask getty.target
```

5.3 CAN FD driver

The CAN FD kernel module must be build from source and signed to be used in a secure boot environment. DKMS can be used to help with building such signed kernel module and maintain it at every kernel update. To install all the required build tools and dependencies:

```
$ sudo apt install gcc make dkms linux-headers-amd64 libelf-dev libpopt
↪ -dev
```

5.3.1 Set-up a key for a signed kernel module

Create a Machine Owner Key Instructions on how to create a custom MOK can be found at the [Debian secure boot wiki](#).

```
$ sudo mkdir -p /var/lib/shim-signed/mok/
$ sudo cd /var/lib/shim-signed/mok/
$ sudo openssl req -nodes -new -x509 -newkey rsa:2048 -keyout MOK.priv
↪ -outform DER -out MOK.der -days 36500 -subj "/CN=My_Name/"
$ sudo openssl x509 -inform der -in MOK.der -out MOK.pem
```

Enrolling your key Key enrollment requires issuing the request using mokutil and then confirming enrollment during a reboot. This is intended to ensure that only the device custodian can approve enrollment of a new MOK.

To enroll a key:

```
$ sudo mokutil --import /var/lib/shim-signed/mok/MOK.der # prompts for
↪ one-time password
```

At next reboot, the device firmware should launch it's MOK manager and prompt the user to review the new key and confirm it's enrollment, using the one-time password. Any kernel modules (or kernels) that have been signed with this MOK should now be loadable.

Adding your key to DKMS In order for dkms to automatically sign kernel modules, it must be told which key to sign the module with. This is done by adding two configuration values to `/etc/dkms/framework.conf`, adjusting paths as required:

```
mok_signing_key="/var/lib/shim-signed/mok/MOK.priv"
mok_certificate="/var/lib/shim-signed/mok/MOK.der"
```

5.3.2 build a signed kernel module

Download [PCAN Linux driver](#) and follow the [build instructions](#) applying the dkms option.

```
$ wget https://www.peak-system.com/quick/PCAN-Linux-Driver
$ mv PCAN-Linux-Driver PCAN-Linux-Driver.tar.gz
$ tar -xzf PCAN-Linux-Driver.tar.gz
$ cd peak-linux-driver-8.18.0/
$ make clean
$ make netdev
$ cd driver/
$ sudo make install_with_dkms
```

5.4 Webmin

Curl can be applied to download the repository installation file.

```
$ curl -o setup-repos.sh https://raw.githubusercontent.com/webmin/
  ↳ webmin/master/setup-repos.sh
```

After download, the installation script can be executed with:

```
$ sudo sh setup-repos.sh
```

5.4.1 Install the Webmin software packages

Now the Webmin repository is added, the Webmin packages can simply be installed with:

```
$ sudo apt install --install-recommends webmin
```

After installation the industrial computer".

5.4.2 Use Webmin

With the industrial computer rebooted, open a browser on the work station and enter the industrial computer's IP address followed by port number 10000.

6. Install Configuration and additional installation

6.1 Configuration of the network devices

6.1.1 Network device 1

To make the configuration more consistent it is recommended to move the stanza of enp1s0 in /etc/network/interfaces to a separate file "/etc/network/interfaces.d/enp1s0.conf"

```
auto enp1s0
iface enp1s0 inet dhcp
```

For later references a comment can be placed in "/etc/network/interfaces"

```
# The primary network interface
# configuerd in "/etc/network.interfaces.d/enp1s0.conf"

# The NetCat interface
# configuerd in "/etc/network.interfaces.d/enp2s0.conf"

# The Battcash interface
# configuerd in "/etc/network.interfaces.d/enp3s0.conf"
```

6.1.2 Network device 2

This network device will be configured via the CodeSys User Interface. On the Industrial PC side it can be enabled with

```
$ sudo ip link set enp2s0 up
```

To bring up the interface permanent create "/etc/network/interfaces.d/enp2s0.conf" and put in the following content

```
auto enp2s0
iface enp2s0 inet manual
```

6.1.3 Network device 3

```
$ sudo ip addr add 192.168.1.2/24 dev enp3s0
$ sudo ip link set enp3s0 up
```

To bring up the interface permanent create `"/etc/network/interfaces.d/enp3s0.conf"` and put in the following content

```
auto enp3s0
iface enp3s0 inet static
address 192.168.1.2/24
```

6.1.4 CAN FD 0

```
$ sudo ip link set can0 txqueuelen 65536
$ sudo ip link set can0 up type can fd on tq 12 prop-seg 63 phase-seg1
    → 64 phase-seg2 32 sjw 32 dtq 12 dprop-seg 15 dphase-seg1 16 dphase
    → -seg2 8 dsjw 8 restart-ms 1000 fd on
```

To bring up the interface permanent create `"/etc/network/interfaces.d/can0.conf"` and put in the following content

```
auto can0
iface can0 inet manual
post-up /sbin/ip link set can0 txqueuelen 65536
```

To configure the CAN specific options, `"systemd-networkd"` should be enabled and configured. This can be done with the following commands

```
$ sudo systemctl start systemd-networkd
$ sudo systemctl enable systemd-networkd
```

To configure set up the CAN specific options create `"/etc/systemd/network/can0.network"` and put in the following content

```
[Match]
Name=can0

[CAN]
BitRate=500K
SamplePoint=80%
TimeQuantaNSec=12
PropagationSegment=63
PhaseBufferSegment1=64
PhaseBufferSegment2=32
SyncJumpWidth=31
DataBitRate=2000K
DataSamplePoint=80%
```

```
DataTimeQuantaNSec=12
DataPropagationSegment=15
DataPhaseBufferSegment1=16
DataPhaseBufferSegment2=8
DataSyncJumpWidth=8
FDMode=True
RestartSec=1000ms
```

More details about the systemd-networkd configuration can be found at [systemd-networkd configuration options](#)

6.1.5 CAN FD 1

```
$ sudo ip link set can1 txqueuelen 65536
$ sudo ip link set can1 up type can fd on tq 12 prop-seg 63 phase-seg1
    ↪ 64 phase-seg2 32 sjw 32 dtq 12 dprop-seg 15 dphase-seg1 16 dphase
    ↪ -seg2 8 dsjw 8 restart-ms 1000 fd on
```

To bring up the interface permanent create `"/etc/network/interfaces.d/can1.conf"` and put in the following content

```
auto can1
iface can1 inet manual
post-up /sbin/ip link set can1 txqueuelen 65536
```

To configure set up the CAN specific options create `"/etc/systemd/network/can1.network"` and put in the following content

```
[Match]
Name=can1

[CAN]
BitRate=500K
SamplePoint=80%
TimeQuantaNSec=12
PropagationSegment=63
PhaseBufferSegment1=64
PhaseBufferSegment2=32
SyncJumpWidth=31
DataBitRate=2000K
DataSamplePoint=80%
DataTimeQuantaNSec=12
DataPropagationSegment=15
DataPhaseBufferSegment1=16
DataPhaseBufferSegment2=8
DataSyncJumpWidth=8
FDMode=True
```

```
RestartSec=1000ms
```

6.2 CodeSys

Use the CodeSys UI from a Windows machine. After reboot of the Industrial PC, CodeSys will be using tty1. Apply ctrl + alt + 2 to switch to tty2 and log on.

6.3 Cyphal

6.3.1 Python 3

```
$ sudo apt install python3-numpy python3-full python3-pip can-utils  
→ pipx
```

6.3.2 PyCyphal

Note: When using sudo pipx will install for the root user.

```
$ pipx install "pycyphal[transport-can-pythoncan,transport-serial,  
→ transport-upd]" --include-deps  
$ pipx install yakut  
$ pipx ensurepath  
$ mkdir ~/dsdl
```

Environmental variables Export the following variables:

```
$ export CYPHAL_PATH="$HOME/dsdl"  
$ export YAKUT_COMPILE_OUTPUT="$~/yakut"  
$ export YAKUT_PATH="$YAKUT_COMPILE_OUTPUT"  
$ export UAVCAN__CAN__IFACE="socketcan:can0":"socketcan:can1" # Must  
→ match YOUR socketcan device name!  
$ export PYTHONPATH="$HOME/dsdl:$HOME/.local/pipx/venvs"
```

These lines can also be added to "~/.bashrc" for automated execution each time a bash shell is started.

6.3.3 PyDsdl

This is a python library that also needs to be install in a virtual environment to prevent breaking the O.S. Because it's a library pipx cannot be used, so we must use pip from a virtual environment in stead. To create a new virtual environment for PyDsdl:

494
495
496

```
$ python3 -m venv ~/.local/pipx/venvs/pydsdl
```

497 To install PyDsdl pip can be called from this virtual environment:

498
499
500

```
$ ~/.local/pipx/venvs/pydsdl/bin/pip install pydsdl
```